

Project 2: Fun with Filters and Frequencies

Alexander Tsai

Part 1.1: Convolutions from Scratch

We implemented two versions of convolution using only numpy. The first uses four for loops: two over the image pixels and two over the filter. The second uses two for loops: for each pixel we take the patch of the image under the filter, multiply it by the filter, and sum.

Both versions flip the filter first, since that is the difference between convolution and cross-correlation. Both also pad the image with zeros, by half the filter size on each side, so the output has the same size as the input. The box filter is a 9x9 array of ones divided by 81, so its weights add up to one.

```
def pad(image, pad_h, pad_w):
    h, w = image.shape
    new_h = h + 2 * pad_h
    new_w = w + 2 * pad_w
    padded = np.zeros((new_h, new_w))
    padded[pad_h : pad_h + h, pad_w : pad_w + w] = image
    return padded

def convolve_4loop(image, filter):
    filter = np.flip(filter) # Ensure the filter is flipped.
    pad_h, pad_w = filter.shape[0] // 2, filter.shape[1] // 2
    padded_image = pad(image, pad_h, pad_w)
    output = np.zeros(image.shape)
    for row_idx in range(image.shape[0]):
        for column_idx in range(image.shape[1]):
            for f_h_idx in range(filter.shape[0]):
                for f_w_idx in range(filter.shape[1]):
                    output[row_idx, column_idx] += filter[f_h_idx, f_w_idx] * padded_image[row_idx + f_h_idx, column_idx + f_w_idx]
    return output

def convolve_2loop(image, filter):
    filter = np.flip(filter) # Ensure the filter is flipped.
    pad_h, pad_w = filter.shape[0] // 2, filter.shape[1] // 2
    padded_image = pad(image, pad_h, pad_w)
    output = np.zeros(image.shape)
    for row_idx in range(image.shape[0]):
        for column_idx in range(image.shape[1]):
            mini_instance = padded_image[row_idx:(row_idx + filter.shape[0]), column_idx:(column_idx + filter.shape[1])]
            output[row_idx, column_idx] = np.sum(mini_instance * filter)
    return output
```

Method	Runtime (9x9 box, 500x375 image)	Max difference from scipy
Four loops	2.317 s	1.0e-15
Two loops	0.196 s	6.7e-16
scipy.signal.convolve2d	0.0103 s	-

There is basically no difference from the SciPy output. The largest difference in any pixel is about 1e-15, which is just floating point error. However, SciPy is much faster. The two-loop version is about 12 times faster than the four-loop version, and SciPy is about 19 times faster than the two-loop version.

For the boundary, we fill with zeros, which is also what `scipy.signal.convolve2d` does by default with `mode="same"`. That is why the two outputs match. One side effect is that the box filter result gets a little darker right at the border, since it averages in the zeros.



Original



9x9 box filter



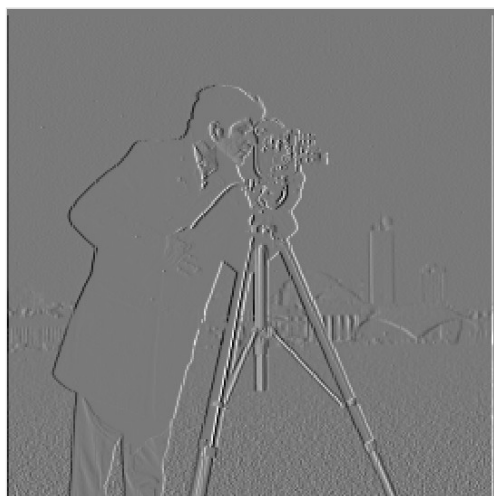
Dx



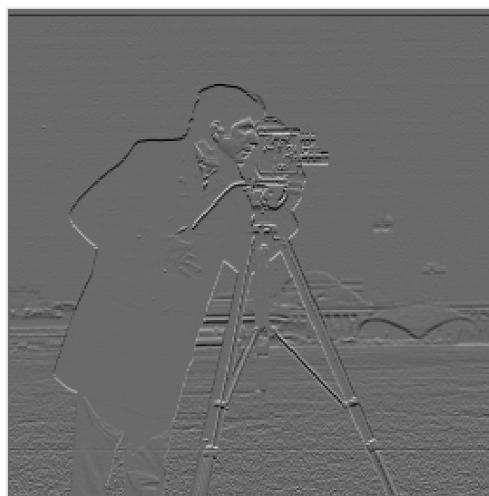
Dy

Part 1.2: Finite Difference Operator

```
Dx = np.array([[1, 0, -1]])
Dy = Dx.T
box = np.ones((9, 9)) / 81
```



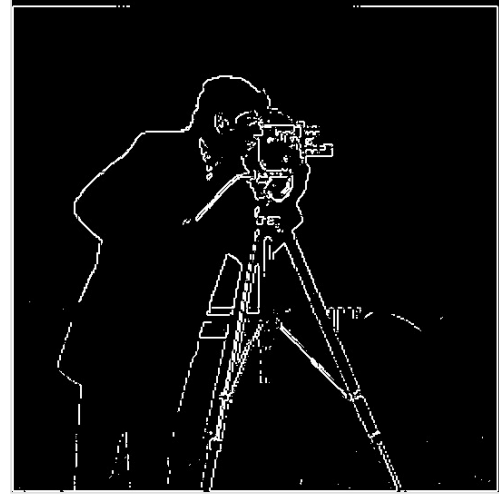
Partial derivative in x



Partial derivative in y



Gradient magnitude



Binarized, threshold 0.33

We convolved the cameraman with D_x and D_y to get the partial derivatives in x and y . The gradient magnitude at each pixel is the square root of $(dx^2 + dy^2)$. It is large wherever the brightness changes quickly, which is where the edges are.

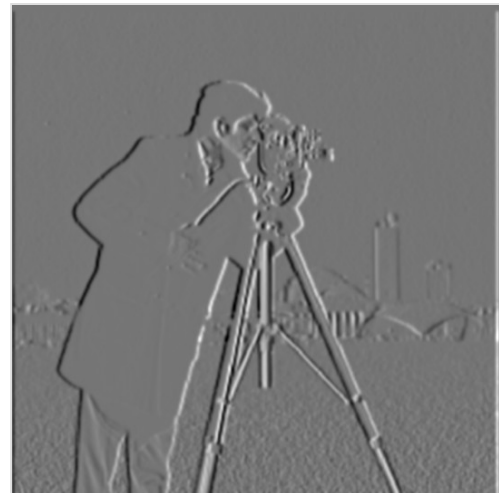
We tried thresholds from 0.1 to 0.33 and ultimately decided on 0.33. That is where the noise from the grass almost completely disappears, yet the outline of the person and the camera still more or less stays intact. Lower thresholds keep more of the edges but also keep a lot of the grass.

Part 1.3: Derivative of Gaussian (DoG) Filter

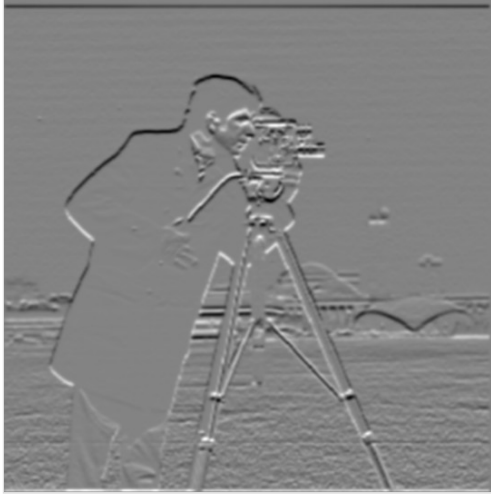
Blur, then finite difference



Blurred, 9x9 Gaussian, sigma 1.5



Partial derivative in x

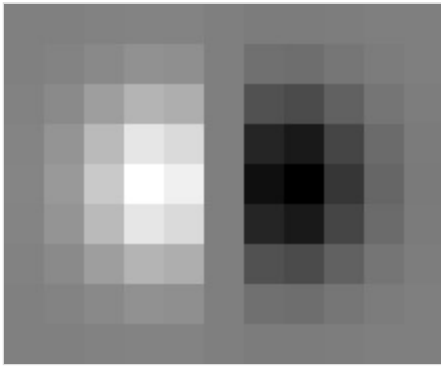


Partial derivative in y

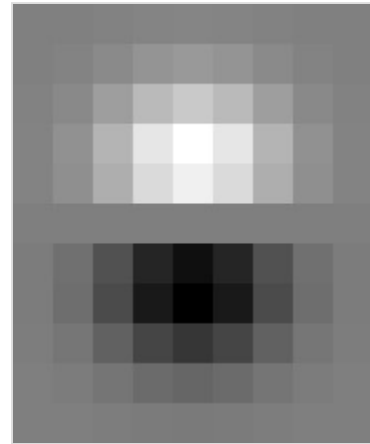


Gradient magnitude

DoG filters

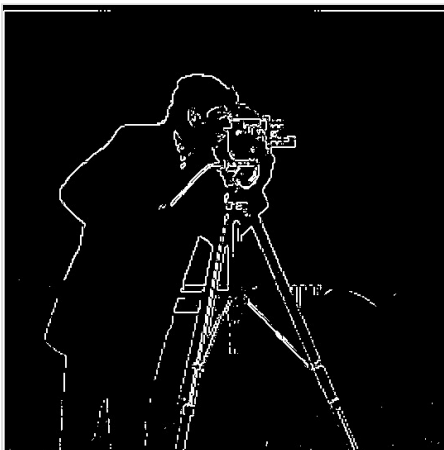


DoG x filter



DoG y filter

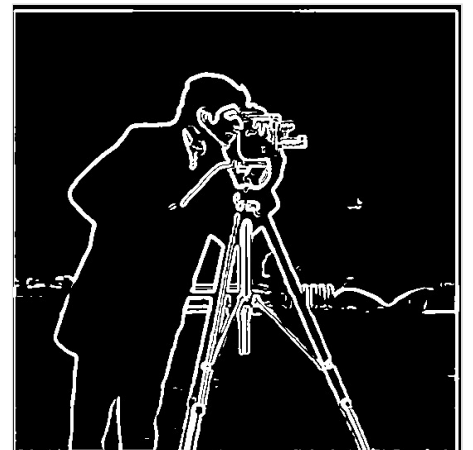
Edges



Finite difference, threshold 0.33



Blur then difference, threshold 0.1



DoG, threshold 0.1

Largest difference between the two gradient magnitudes (blur then difference versus one DoG convolution): $7.6e-16$.

We created the Gaussian filter using `cv2.getGaussianKernel` (size 9, sigma 1.5) and multiplying it by its transpose to get a 2D filter. We blurred the cameraman with it and then repeated Part 1.2.

There are two main differences. One, the result is far less noisy. Two, the edges are far clearer and more continuous. This is because taking a derivative amplifies small pixel-to-pixel noise, and blurring first removes that noise before we take the derivative. The edges also come out a bit thicker, since the blur spreads them out. Because the blurred gradients are weaker, we could use a lower threshold of 0.1.

The DoG filters do the same thing, but in one step. We convolve the Gaussian with D_x and D_y first, and then convolve the image once with each result. Convolution is associative, so we can group the filters this way. We found that the largest difference between the two gradient magnitudes is $7.6e-16$ (rather small). So they match.

```
kernel_part = cv.getGaussianKernel(ksize=9, sigma=1.5)
G = kernel_part @ kernel_part.T

DoG_x = convolve2d(G, Dx)
DoG_y = convolve2d(G, Dy)
gx_dog, gy_dog, mag_dog = gradient_magnitude(cameraman, DoG_x, DoG_y)
```

Part 2.1: Image Sharpening

We blur the image with a Gaussian filter (size 9, sigma 1.5), which keeps the low frequencies. Subtracting the blurred version from the original gives us the high-frequency details. We then add those details back to the original, scaled by alpha, to sharpen it. A bigger alpha means more sharpening.

```
G = gaussian_kernel(ksize=9, sigma=1.5)
taj_blur = filter_color(taj, G)
high = taj - taj_blur
sharpened_taj = taj + alpha * high
```

We can also do this in one convolution. The filter is $(1 + \alpha)$ times the identity filter minus α times the Gaussian. The identity filter just keeps the original pixel. This gives the same result as subtracting the blur and adding back the details.

```
def unsharp_mask(image, alpha, gaussian):
    impulse = np.zeros_like(gaussian)
    impulse[gaussian.shape[0] // 2, gaussian.shape[1] // 2] = 1
    kernel = (1 + alpha) * impulse - alpha * gaussian
    return filter_color(image, kernel)
```

Taj Mahal



Original



Blurred, sigma 1.5



High frequencies, rescaled



Sharpened, alpha 1



alpha 1

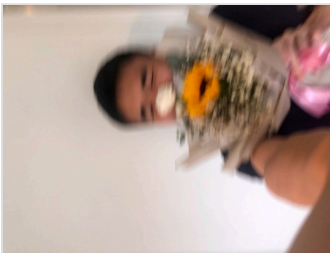


alpha 2

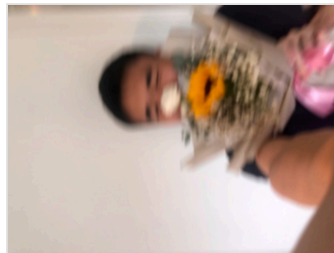


alpha 5

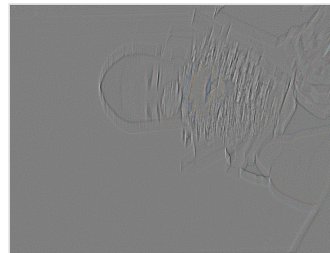
My own blurry image



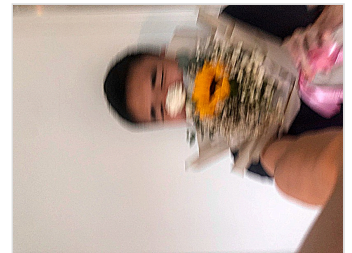
Original



Blurred, sigma 1.5



High frequencies, rescaled



Sharpened, alpha 5



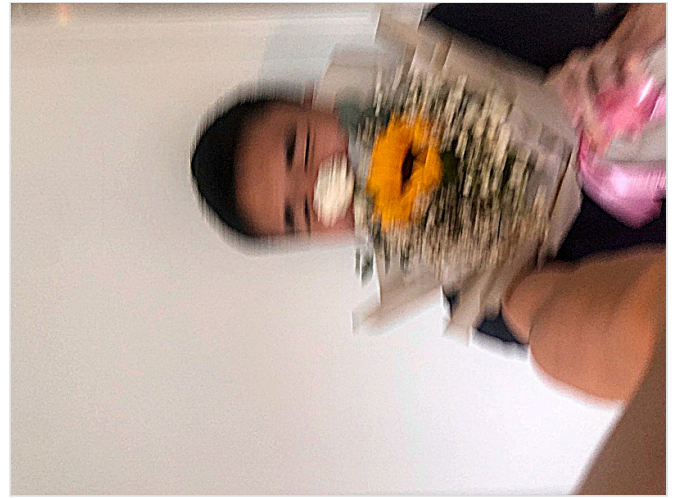
Original



alpha 1



alpha 2



alpha 5

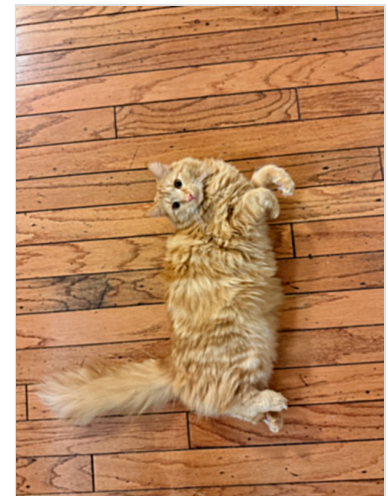
Blur a sharp image, then sharpen it again



Original



Blurred



Re-sharpened, alpha 2

For the Taj, alpha around 1 to 2 looked the sharpest without distorting the image. Alpha values that were too high resulted in distorted-looking images with harsh edges and noise. For my own blurry image, I chose an alpha of 5 despite it not being perfect.

For the last test we took a sharp image, blurred it, and sharpened it again. The re-sharpened image looks better than the blurred one, but it still does not match the original. The blur weakens the finest details, and sharpening can only boost the details that are still there. This unsharp mask does not undo the blur.

Part 2.2: Hybrid Images

For each hybrid, we keep the broad shapes from one image and the fine details from the other. A Gaussian keeps the low frequencies. Subtracting a Gaussian blur from an image gives its high frequencies. The sigmas below are in pixels. A larger sigma means a lower frequency cutoff.

Hybrid 1: Derek + Nutmeg



Derek, original



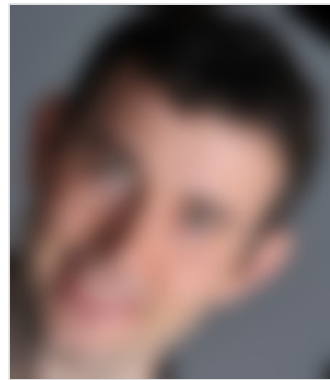
Nutmeg, original



Derek, aligned



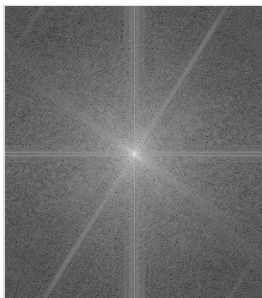
Nutmeg, aligned



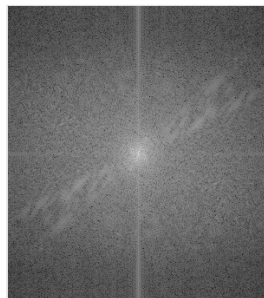
Derek low-pass, sigma 10



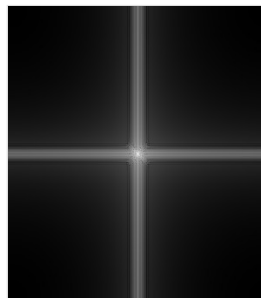
Nutmeg high-pass, sigma 4



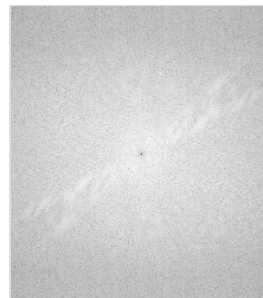
FFT: Derek



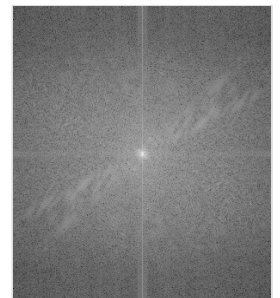
FFT: Nutmeg



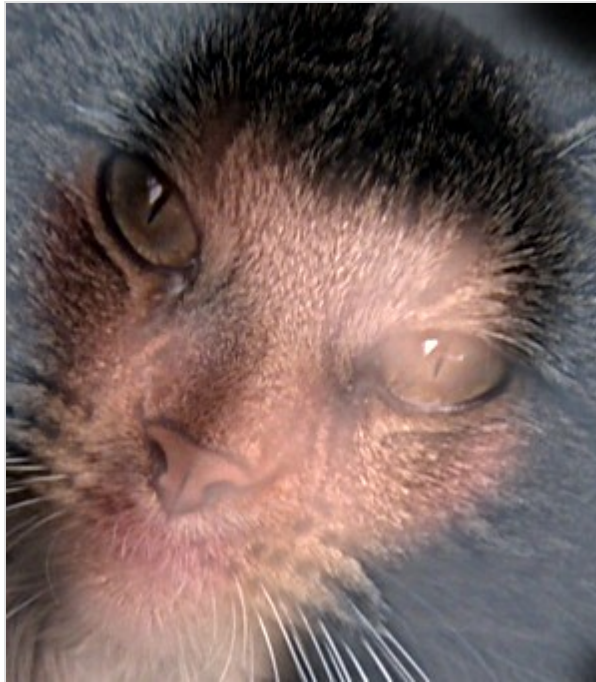
FFT: Derek low-pass



FFT: Nutmeg high-pass



FFT: hybrid



Hybrid

We first struggled with aligning Derek and Nutmeg. We used the provided alignment code and clicked on the eyes in both images, then cropped both to the same region. Once aligned, we low-pass filtered Derek and high-pass filtered Nutmeg (the image minus its blurred version), and added the two together.

To pick the cutoffs, we tried a grid of sigma values for both images and looked at each result up close and from far away. We picked sigma 10 for Derek and sigma 4 for Nutmeg. Up close you mostly see the cat, and from far away you mostly see Derek.

```
def low_pass(image, sigma):
    ksize = int(6 * sigma) | 1
    return filter_color(image, gaussian_kernel(ksize, sigma))

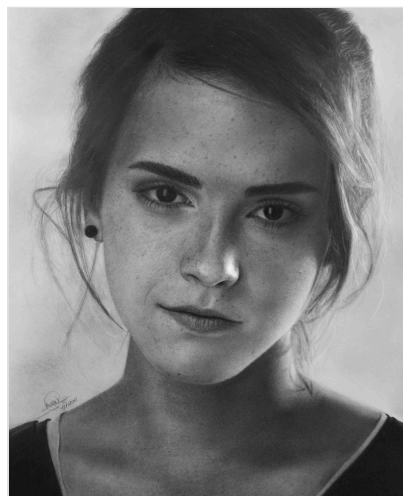
def high_pass(image, sigma):
    return image - low_pass(image, sigma)

def hybrid_image(im_low, im_high, sigma_low, sigma_high):
    return low_pass(im_low, sigma_low) + high_pass(im_high, sigma_high)
```

Hybrid 2: selfie + Emma Watson drawing



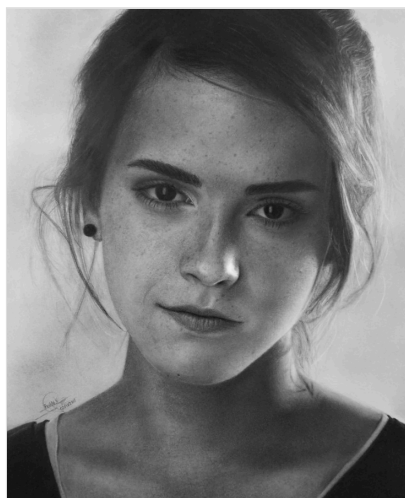
Selfie, original



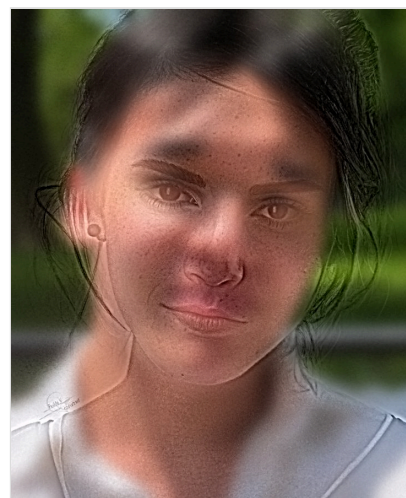
Drawing, original



Selfie, aligned; low-pass sigma 12



Drawing, aligned; high-pass sigma 4



Hybrid

One issue here is that the color schemes of the two images are quite different. The green background of the selfie stands out against the black and white drawing. The drawing also has weaker contrast, so its high frequencies are harder to see. We used sigma 12 for the selfie and sigma 4 for the drawing, and multiplied the high frequencies by 1.5 to make them more visible.

Hybrid 3: cruise (favorite, full process)



Photo 1, original

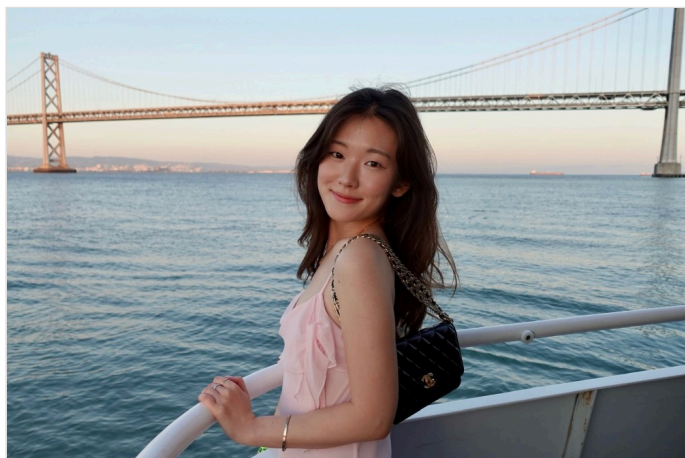


Photo 2, original

This one is my favorite because I really enjoyed this cruise that I went on. We aligned matching points on the faces using the provided code. We tried low-pass sigmas of 2, 3, 4, and 6, and high-pass sigmas of 1, 1.5, 2, and 3. We picked 4 and 2. The faces are smaller here than in the Derek and Nutmeg pair, so smaller sigmas worked better. The different head poses still make the switch less clear, even though I like the photos.



Photo 1, aligned

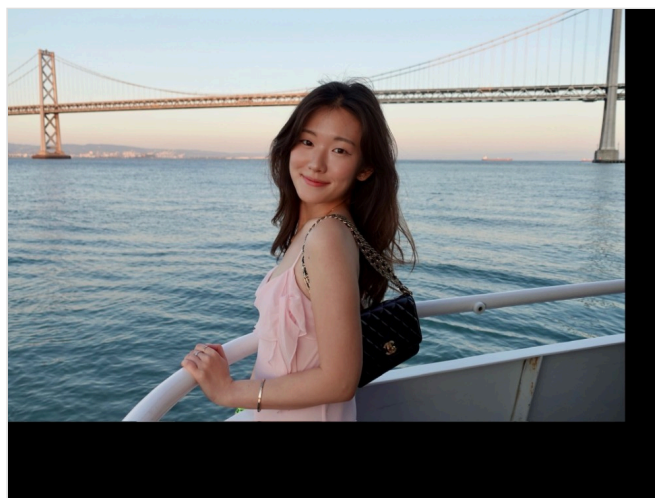


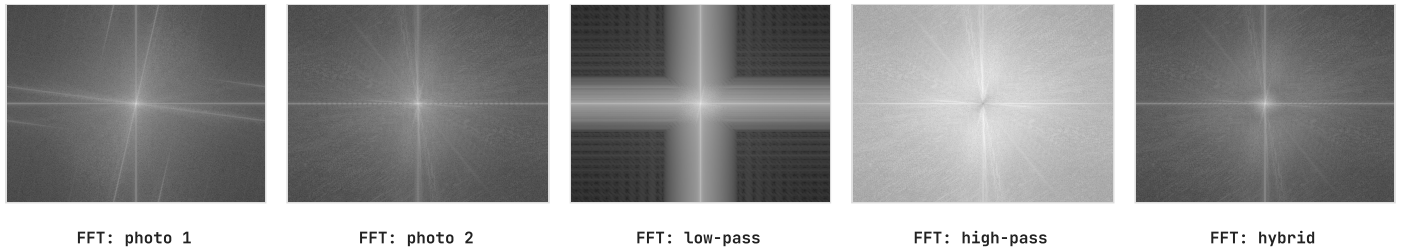
Photo 2, aligned



Photo 1 low-pass, sigma 4



Photo 2 high-pass, sigma 2



These are the log magnitudes of the Fourier transforms, with the zero frequency moved to the center. The low-pass filter reduces the frequencies further from the center. The high-pass filter reduces the center and keeps more of the outer detail. The hybrid combines both. Each plot is rescaled for display, so the brightnesses are not directly comparable between plots.

There is also a cross pattern in the FFTs. These photos have a lot of horizontal and vertical features, such as the waterline and the buildings. Those features help create strong lines along the frequency axes. The image boundaries can contribute too. Photo 1 was rotated to get the alignment right, so some of its frequency lines are tilted as well. The hybrid combines both photos, so it has both tilted and straight lines, although the tilted ones are less obvious after filtering.



Hybrid

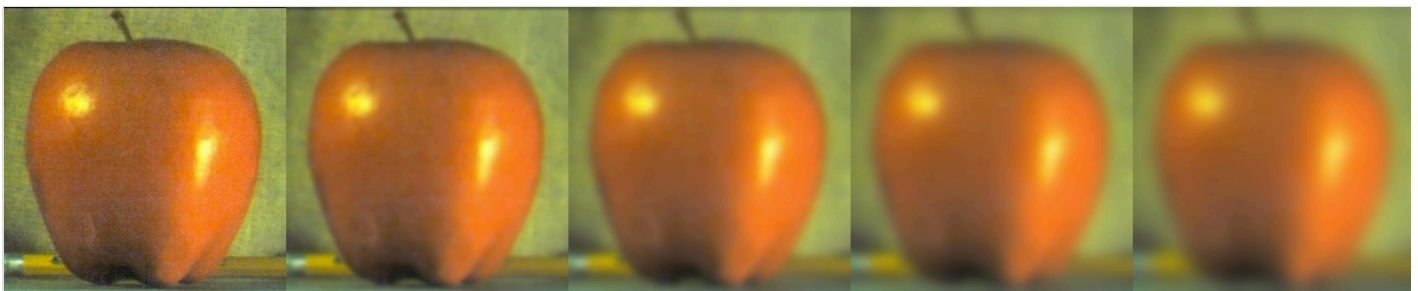
Part 2.3: Gaussian and Laplacian Stacks

We implemented the Gaussian and Laplacian stacks with simple for loops. The Gaussian stack starts with the original image, and each level after that is the original blurred with a bigger sigma (2, 4, 6, 8).

Each Laplacian level is $L[i] = G[i] - G[i + 1]$. That leaves the details between two Gaussian levels. The last level is $L[4] = G[4]$, the blurriest Gaussian level. Adding up all the Laplacian levels gives back the original image.

```
def gaussian_stack(image, levels, sigma):
    g_stack = [image]
    for level in range(1, levels):
        new_sigma = level * sigma
        g_stack.append(low_pass(image, new_sigma))
    return g_stack

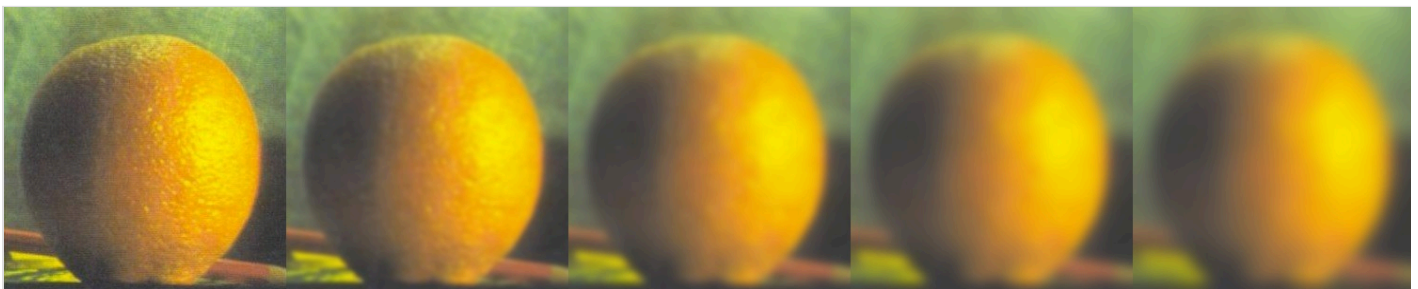
def laplacian_stack(g_stack):
    l_stack = []
    for i, layer in enumerate(g_stack[:-1]):
        l_stack.append(layer - g_stack[i+1])
    l_stack.append(g_stack[-1])
    return l_stack
```



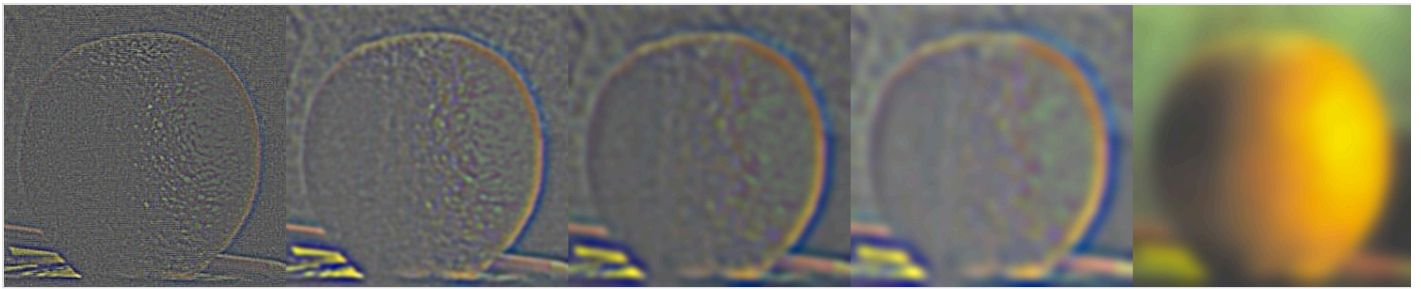
Apple, Gaussian stack, levels 0 to 4



Apple, Laplacian stack, levels 0 to 4

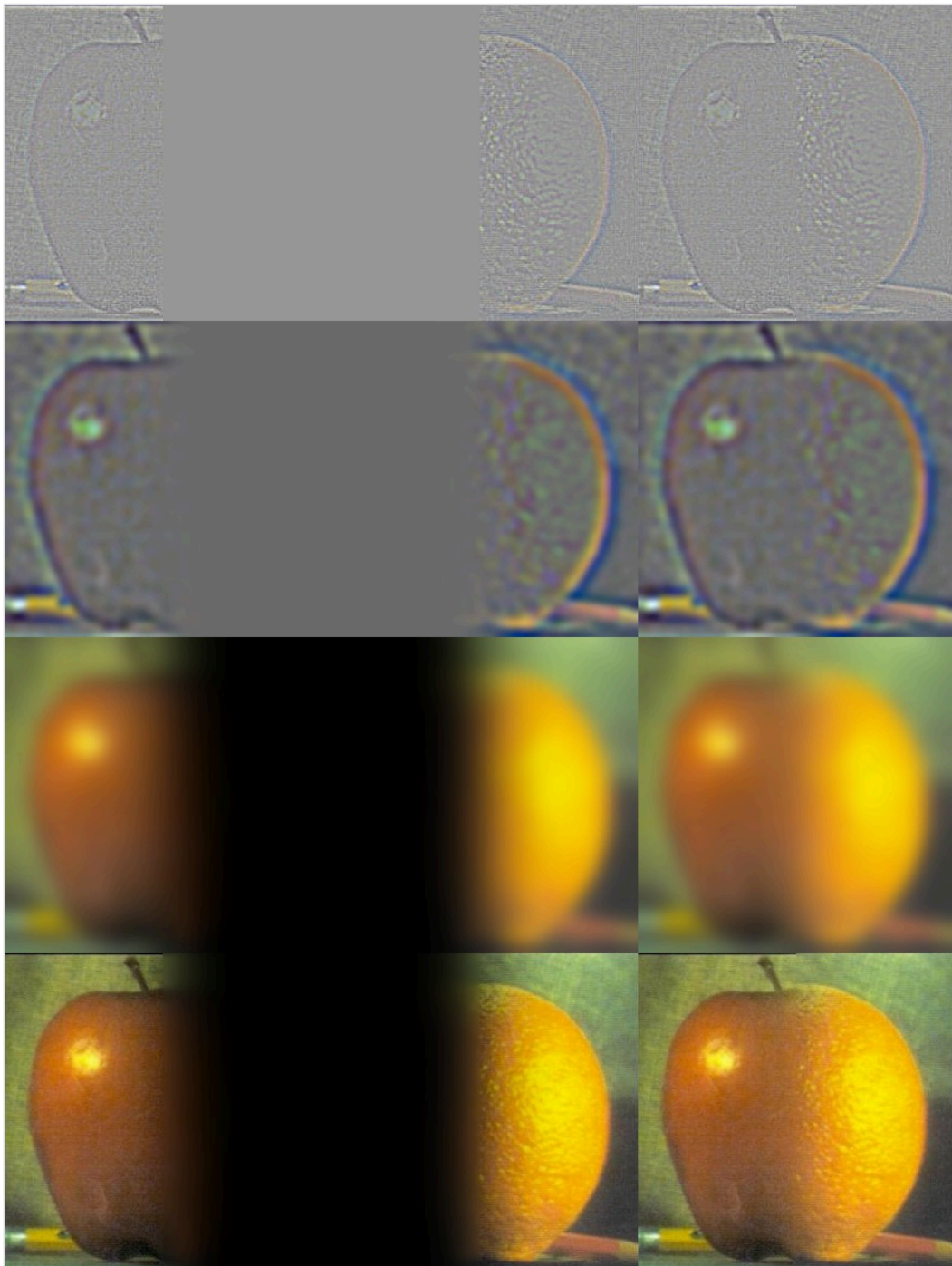


Orange, Gaussian stack, levels 0 to 4



Orange, Laplacian stack, levels 0 to 4

Recreation of Szeliski Figure 3.42



Columns: masked apple, masked orange, sum. Rows: Laplacian levels 0, 2, 4, then all levels summed. Reading across each row gives (a)-(c), (d)-(f), (g)-(i), and (j)-(l) from Figure 3.42. Level 4 is the coarse residual. Detail rows are rescaled together for display.

Part 2.4: Multiresolution Blending

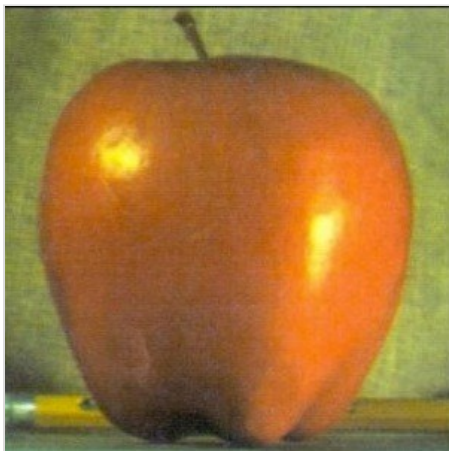
If you just cut two images in half and stick them together, you get a hard line. If you fade between them over a wide area, the fine details of both images overlap and look like a ghost. The fix is to blend each level of detail separately. Fine details should switch over a narrow area, and the broad colors should fade over a wide area.

So we build a Laplacian stack for each image and a Gaussian stack for the mask. The mask gets blurrier at each level, which makes the fade wider for the coarser levels. At each level we take mask times the first image plus (1 minus mask) times the second image. Then we add up all the levels to get the final image.

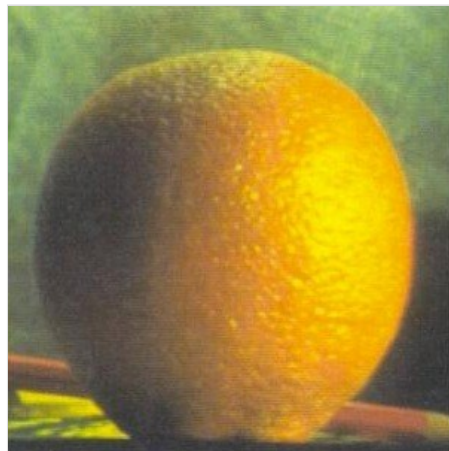
We tried mask sigmas from 1 to 8 and ended up choosing 8, as that most closely replicated the orapple in Figure 3.42. Smaller values still showed a line down the middle.

```
def blend(im_a, im_b, mask, levels, sigma, mask_sigma):
    l_a = laplacian_stack(gaussian_stack(im_a, levels, sigma))
    l_b = laplacian_stack(gaussian_stack(im_b, levels, sigma))
    g_mask = gaussian_stack(mask, levels, mask_sigma)
    l_blend = []
    for i in range(levels):
        l_blend.append(l_a[i] * g_mask[i] + l_b[i] * (1 - g_mask[i]))
    return np.sum(l_blend, axis = 0), l_a, l_b, g_mask
```

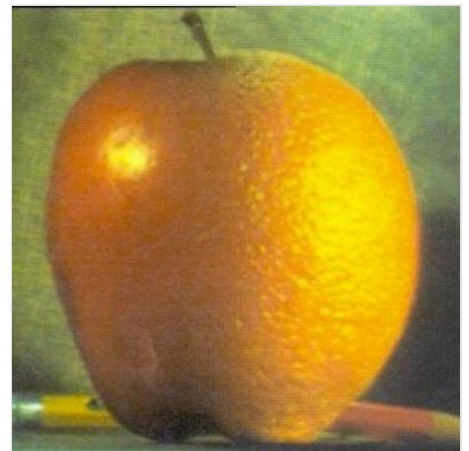
Orapple



Apple

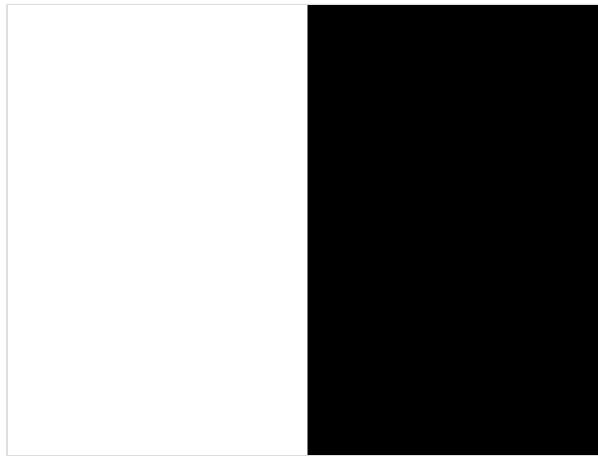


Orange



Orapple, mask sigma 8

Custom blend 1 (straight seam)



Step mask



Left image



Right image



Blend

This is my favorite result. The two photos were taken on the same snowy road, so the colors and lighting already match. The seam is somewhat obvious in the road. But if you look outside the road area, the two scenes actually merge together pretty well.

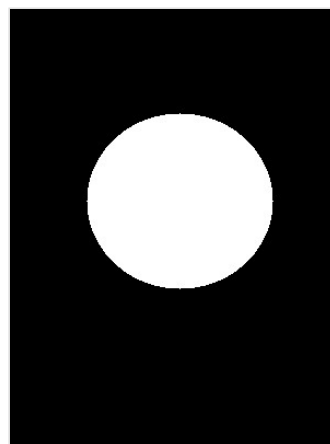
Custom blend 2 (irregular mask)



Tree at night



Tree in daytime



Ellipse mask

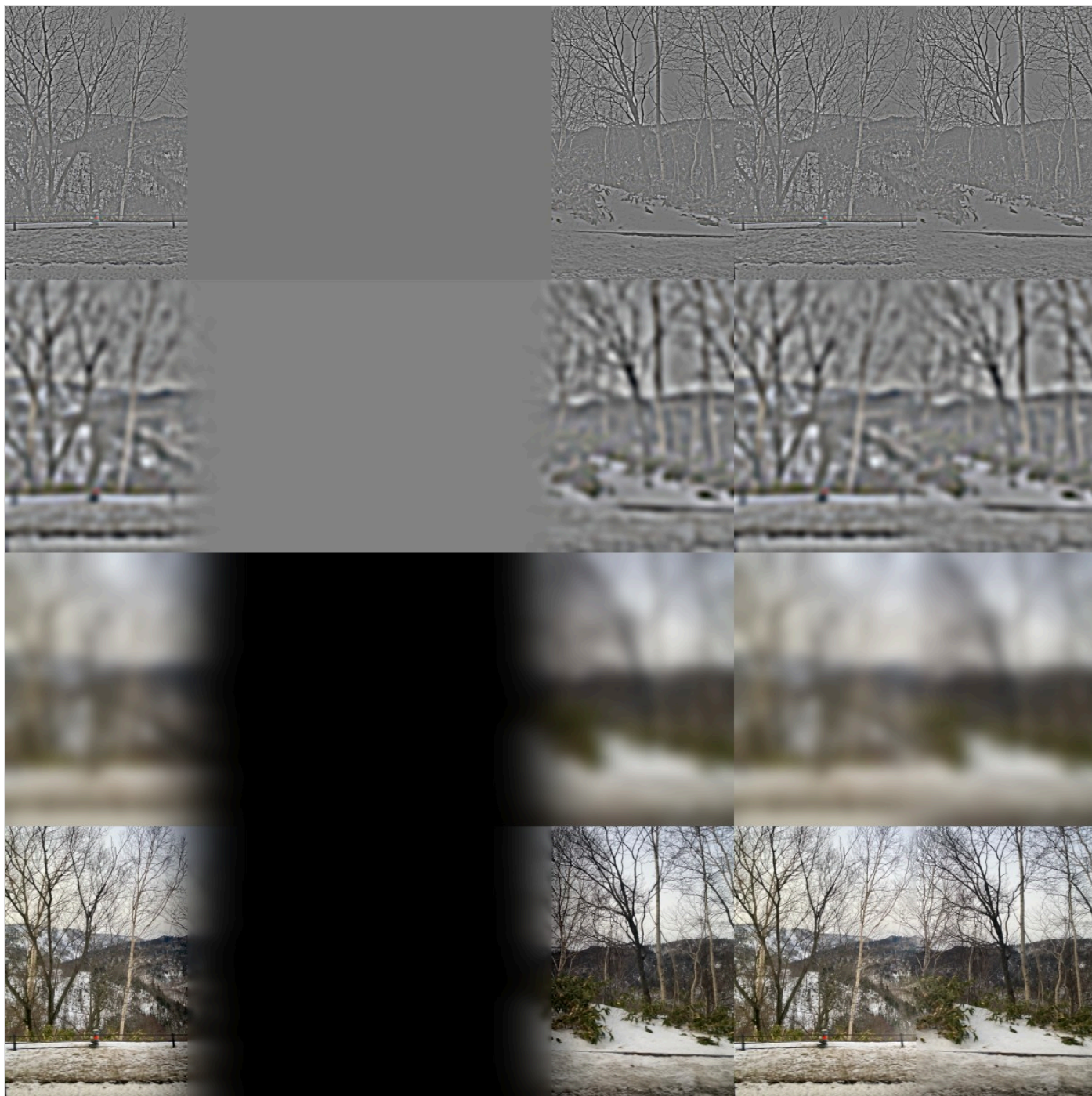


Blend

The tree blend went pretty strangely. It was honestly a semi-failure. We tried to place the tree lit up at night into the daytime photo using an ellipse as the mask. The problem is that the two photos are too different. The night photo is very dark around the tree, so a dark rim shows up around the ellipse, and the two trees do not line up well. Blending works much better when the two images are already similar near the seam.

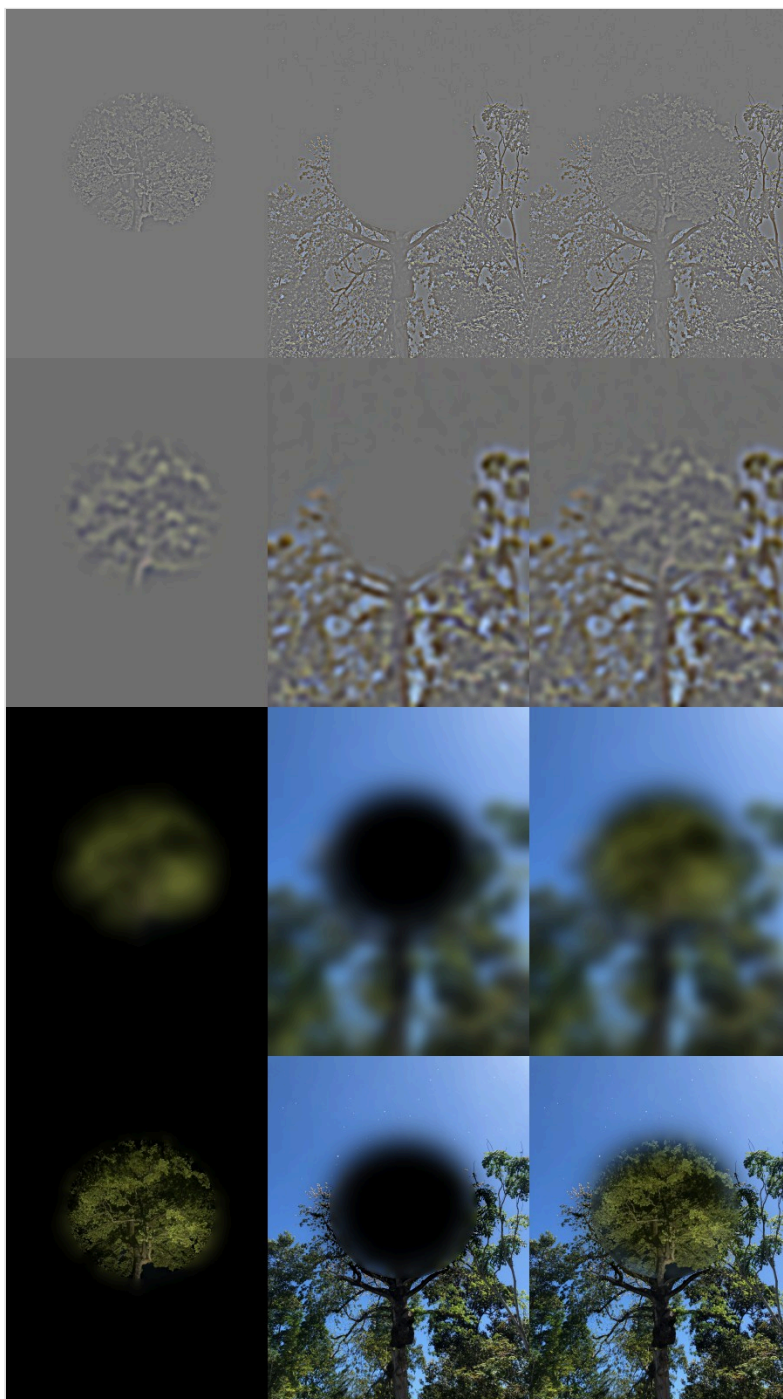
Laplacian stacks for my favorite result (custom blend 1)

This is laid out like Figure 10 in the Burt and Adelson paper. The first three rows show levels 0, 2, and 4, from fine to coarse. The left column is the first image after the mask, the middle column is the second image after the mask, and the right column is the two added together. The last row adds up all the levels, which gives the final blend. You can see the fade between the two images get wider as the levels get coarser.



Columns: masked left image, masked right image, sum. Rows: Laplacian levels 0, 2, 4, then all levels summed.

Laplacian stacks for custom blend 2



Columns: masked night tree, masked day tree, sum. Rows: Laplacian levels 0, 2, 4, then all levels summed.

What I Learned

The most important thing I learned is that seemingly very simple ideas, such as splitting an image into Gaussian and Laplacian layers, can produce really powerful results. In a world where we are so focused on generating images with AI, it is honestly quite surprising that these traditional methods can still produce such funny and fascinating-looking things.

References: [Project 2 assignment](#), [Burt and Adelson \(1983\)](#). Derek, Nutmeg, the cameraman, the Taj Mahal, the apple, the orange, and the alignment helper came from the course materials.